

DCA-01 - Deterministic Cluster Architecture

Vollständiges technisches Dossier - Baufertige Spezifikation

Vom monolithischen Modell zum deterministischen KI-Ökosystem

Version: 1.2 COMPLETE DOSSIER

Autor: Emanuel Schaaf

Datum: 2026-10-06

Status: Implementierbar - Alle Schemata enthalten

Vorlage: Just-in-Sequence Fertigung (Automotive) + Triple-Modular-Redundancy TMR (Avionik) + CAN-Bus / Blackboard + LRU Prinzip

DCA-01 | DETERMINISTIC CLUSTER ARCHITECTURE

DATENFLUSS: VERTIKAL | 2-AUS-3 VOTING | DELTA-PRÜFUNG | RL SELF-OPTIMIZATION

USER REQUEST

Eingehende Anfrage • Auth & Rate-Limit • Eingabevalidierung

TMR ROUTER CLUSTER — 2-AUS-3 VOTING

ROUTER A

Routing • Prio-Queue A
• Timeout: 100ms

ROUTER B

Routing • Prio-Queue B
• Timeout: 100ms

ROUTER C

Routing • Prio-Queue C
• Timeout: 100ms

VOTING-LOGIK: 2-AUS-3

Mehrheitsentscheid → Konsistenzprüfung • Fehlerzähler & Byzantine-Filter

TASK DECOMPOSER

Zerlegung in DAG

T_{physik}

{Physikalische Modellierung}

T_{code}

{Code-Generierung / Logik}

T_{check}

{Konsistenz- & Formprüfung}

Abhängigkeiten: T_{physik} → T_{code} → T_{check} • Parallelisierbar

SHARED CONTEXT BOARD / SEMANTIC BUS — BLACKBOARD

Zentraler gemeinsamer Kontextspeicher • Publ./Subscr. • Versionierte Einträge • Semantische Embedding-Indizierung

EXPERTEN-SCHICHT (PARALLEL)

PHYSIK/MATHE EXPERTE HPC

Funktion: Numerik • Simulation
• Physikmodelle

OUTPUT: (y_i)

y_i = Ergebnis + Konfidenz + Metriken

CODE/LOGIK EXPERTE MID-GPU

Funktion: Code-Synthese • Logikprüfung
• Verifikation

OUTPUT: (c_i)

c_i = Code + Tests + Verifikationsstatus

SPRACHE/STRUKTUR EXPERTE — CPU

Funktion: Textstruktur • Semantik
• Zusammenfassung

OUTPUT: (t_i)

t_i = Antwortentwurf + Kohärenz + Quelle

FUSIONS-SCHICHT

Aggregation & Fusion

DELTA-PRÜFUNG

max 3 Iter • Neu-Abfrage bei Δ > Schwelle • Konfliktlösung

AUDIT LOG

Zeitstempel • Entscheidungen • Voting-Log • Modellversion
v1.4.2

PROVENANCE

Datenherkunft • Expertensignatur • Lineage-Trace
• Hash: 0x9F4A...C7D1

RL SELF-OPTIMIZATION LOOP — LoRA

POLICY UPDATE

Reward: Antwortqualität, Latenz, Kosten • PPO/RLHF

OFFLINE TRAINING: Erfahrungsbuffer → LoRA Weight Update • Sicherer Rollout via Canary

FEEDBACK: LoRA-Adaption → Prompt
npt-Optimierung • Strategie-
Update • Parameter-Δ

Inhaltsverzeichnis

1. Executive Summary und Zielbild
 2. Design Prinzipien
 3. Gesamtarchitektur und Datenfluss
 4. Modul 1: TMR Router Cluster - Eliminierung SPOF
 5. Modul 2: Task Decomposer - DAG Generierung
 6. Modul 3: Semantischer Bus - Shared Context Board
 7. Modul 4: Domänen Experten - Isolierte Basis-Modelle
 8. Modul 5: Deterministische Fusions-Schicht
 9. Vollständige Datenmodelle und JSON Schemas
 10. Algorithmen und Pseudocode
 11. Hardware Mapping und Kostenmodell
 12. Selbstoptimierung - RL Loop und Continual Learning
 13. Observability, Logging und Metriken
 14. Sicherheit, Isolation und Signierung
 15. Compliance Mapping EU AI Act
 16. Deployment, Lifecycle und Model Registry
 17. Failure Modes und Recovery
 18. Implementierungs-Roadmap MVP bis Production
 19. Glossar
 20. Anhänge
-

1. Executive Summary und Zielbild

Ziel ist eine KI-Infrastruktur die vier Anforderungen gleichzeitig erfüllt:

- Selbst optimierbar: Jedes Modul lernt aus eigenen Fehlern ohne globales Retraining
- Leichter trainierbar: Lokales LoRA Training, keine globale Re-Zertifizierung
- Genauer und schneller: Nur relevante Experten aktiv, parallel statt seriell, kalibrierte Confidence
- Auditierbar: Jeder Entscheidungspfad geloggt mit Provenance

Statt eines monolithischen Riesenmodells wird ein Cluster aus physisch getrennten, domänenspezialisierten Basis-Modellen über einen deterministischen Datenbus orchestriert.

Kernprinzip: Wissenstransfer findet nicht durch vermischte Gewichte statt, sondern durch getypten Datenaustausch auf einem zentralen Board. Training, Zertifizierung und Hardware-Zuweisung sind lokal pro Knoten. Das Blackbox Problem wird physisch zerschlagen.

Für Meta Muse bedeutet das: Nicht ein großes Llama das alles kann, sondern ein Orchestrator der viele kleine, hochoptimierte Llamas steuert.

2. Design Prinzipien

1. **Harte physische Isolation:** Jedes Fach-Modell hat eigene Gewichte, eigenen Loss, eigene Hardware. Kein Shared Parameterraum.
2. **Determinismus vor Wahrscheinlichkeit:** Bei gleichem Input, gleichem Model Hash und gleichem Seed muss gleicher Output entstehen.
3. **TMR für kritische Pfade:** Routing ist dreifach redundant mit 2-aus-3 Voting und architektonischer Diversität.
4. **Getypter Datenaustausch statt Gewichtsvermischung:** Cross-Domain Transfer über Blackboard Pattern, nicht über gemeinsames Netz.
5. **LRU Prinzip:** Jede Einheit ist Line Replaceable. Austauschbar, rollback fähig, ohne Gesamtsystem Stopp.
6. **Audit by Design:** Jede Entscheidung hat Trace ID, Provenance und signierten Model Hash.

3. Gesamtarchitektur und Datenfluss

```
[Client Request]
|
v
[1. Input Validation + Auth + Prompt Injection Filter]
|
v
[2. TMR Router Cluster Stufe 1 - 3x 100-300M parallel]
|   R_a, R_b, R_c -> Voting -> validated_intent + dag_proposal
v
[3. Task Decomposer Stufe 2 - DAG Builder]
|   T_gesamt -> {T1...Tn} mit Hardware, Budget, Output Spec
v
[4. Shared Context Board / Semantic Bus - Publish/Subscribe]
|   TTL 5000ms, Schema Registry, ACL
+---<---> [Experte Physik/Mathe - HPC H100]
+---<---> [Experte Code/Logik - Mid GPU A6000]
+---<---> [Experte Sprache/Struktur - CPU Cluster]
+---<---> [Experte Ethik/Logik - CPU]
|
v
[5. Fusions Layer - Delta Prüfung + Mikro-Iterationen]
|   E_i(T_i) = (y_i, c_i, t_i, hash, provenance)
|   if delta > 0.25 or type_check fail -> Constraint -> Re-Run nur
fehlerhaftes Modul
v
[6. Finale Antwort + Provenance Graph + Audit Log + Flags]
|   Flags: CONVERGENT, NON_CONVERGENT, REVIEW_REQUIRED,
TMR_DISAGREEMENT
v
[7. Observability + RL Self-Optimization Loop -> LoRA Dataset]
```

Jeder Request bekommt eine `dag_id` als OpenTelemetry Trace ID. Alle Logs referenzieren diese ID.

4. Modul 1: TMR Router Cluster

4.1 Warum TMR

Ein einzelner Router ist Single Point of Failure. Fällt er, fällt alles. In der Avionik sind kritische Steuerungen dreifach redundant. Gleiches hier.

4.2 Router Spezifikation

Drei quantisierte Router mit Diversität in Architektur und Trainingsdaten:

- Router A: Transformer 250M, trainiert auf Intent Klassifikation, Daten Slice A
- Router B: Mamba State-Space 180M, trainiert auf Task Graphen, Daten Slice B
- Router C: Distilled 120M, trainiert nur auf Failure Fällen von A und B, Daten Slice C

Alle Router: quantisiert INT8, Latenz Ziel < 50ms p95, Edge TPU fähig.

Output Format Router:

```
{
  "router_id": "router_a_v1.2",
  "model_hash": "sha256:a1b2c3",
  "intent_label": "code_generation_with_physics",
  "confidence_r": 0.92,
  "dag_proposal": {
    "dag_hash": "sha256:d4e5f6",
    "nodes": ["T_physik", "T_code", "T_check"],
    "edges": [["T_physik", "T_code"], ["T_code", "T_check"]]
  },
  "reasoning_trace": "Anfrage enthält Thermodynamik + Python, daher Split...",
  "latency_ms": 32
}
```

4.3 Voting Protokoll vollständig

```
function tmr_vote(R_a, R_b, R_c):
    if hash(R_a.dag_proposal) == hash(R_b.dag_proposal):
        return majority(R_a, R_b, confidence_weighted)
    if hash(R_a.dag_proposal) == hash(R_c.dag_proposal):
        return majority(R_a, R_c)
    if hash(R_b.dag_proposal) == hash(R_c.dag_proposal):
        return majority(R_b, R_c)

    // total disagreement - kritischer Fall
    validated = max_by_confidence(R_a, R_b, R_c)
    validated.flag = "TMR_DISAGREEMENT"
    validated.flag2 = "REVIEW_REQUIRED"
    log_audit(
        event="tmr_disagreement",
        dag_id=current_dag_id,
        proposals=[R_a,R_b,R_c],
        confidence_spread=max(c)-min(c)
    )
    trigger_dataset_write("router_failures.jsonl", [R_a,R_b,R_c])
    return validated
```

Metriken Router:

- `router_disagreement_rate` Ziel < 5%
- `router_latency_p95` Ziel < 50ms
- `routing_precision` > 95% gemessen gegen Golden Set

Loss Router: $L = 0.6 * CE(intent) + 0.2 * latency_penalty + 0.2 * disagreement_penalty$

4.4 Model Registry Router

Jeder Router versioniert, signiert, mit Training Data Hash. Rollback in < 30s. Registry speichert Eval Report.

5. Modul 2: Task Decomposer

5.1 Aufgabe

Validierten Intent in ausführbaren DAG zerlegen. Decomposer ist selbst kleines LLM 1B mit Planner Fähigkeit, kein Regelwerk.

5.2 Vollständiges Node Schema

```
{
  "node_id": "T_physik_01",
  "dag_id": "dag_20261006_123456",
  "domain": "thermodynamics",
  "objective": "berechne Wärmeübergangskoeffizient für
Rohrbündelwärmetauscher, Wasser/Wasser",
  "input_refs": ["user_request"],
  "target_hardware": "hpc_h100",
  "compute_budget_ms": 800,
  "timeout_ms": 1200,
  "retries": 1,
  "priority": 1,
  "output_spec": {
    "format": "json",
    "schema_ref": "schemas/physik_output_v1.json",
    "required_fields": ["Q_dot", "delta_T", "h_coefficient", "unit"],
    "determinism": {
      "temperature": 0,
      "seed": 42,
      "top_p": 1.0,
      "top_k": 1
    }
  },
  "dependencies": [],
  "blackboard_subscriptions": ["CLOSED_LOOP", "FLOW_PATTERN"]
}
```

Beispiel DAG für "Schreibe Python Simulation für Wärmetauscher":

- T_physik_01: Formelherleitung, keine Dependencies
- T_code_01: Python Implementierung, depends_on T_physik_01, liest dessen Output + Blackboard Pattern
- T_check_01: Unit Test und Typ Check, depends_on T_code_01, T_physik_01

Decomposer erzeugt auch `dag_hash` für Reproduzierbarkeit.

6. Modul 3: Semantischer Bus - Shared Context Board

6.1 Problem und Lösung

Harte Isolation verhindert Catastrophic Forgetting, blockiert aber zufälligen Cross-Domain Transfer. Lösung: Transfer auf Kommunikations-Ebene wie CAN-Bus im Auto.

Vor tiefer Inferenz extrahiert jedes Modul abstrakte Muster und publisht sie.

6.2 Vollständiges Pattern Schema

```
{
  "pattern_id": "CLOSED_LOOP_01",
  "version": "1.0",
  "dag_id": "dag_20261006_123456",
  "source_module": "physik_experte",
  "source_model_hash": "sha256:physik_v2.1",
  "timestamp": "2026-10-06T12:00:00Z",
  "ttl_ms": 5000,
  "pattern_type": "topological",
  "abstraction": {
    "name": "geschlossenes Kreislautsystem",
    "description": "Energieerhaltung, Rückkopplung erforderlich",
    "properties": {
      "conservation": true,
      "feedback_loop": true,
      "steady_state_required": true,
      "suggested_code_pattern": "while_loop_with_convergence"
    }
  },
  "vector_embedding": [0.12, 0.88, 0.34],
  "access_control": {
    "write": ["physik_experte"],
    "read": ["code_experte", "logik_experte", "sprache_experte"]
  },
  "consumers_ack": [],
  "audit": true
}
```

Beispiel Transfer: Physik erkennt geschlossenen Kreislauf. Code liest

suggested_code_pattern: while_loop_with_convergence und wählt while
abs(delta) > eps statt for. Keine physikalischen Gewichte im Code Modell nötig.

6.3 Bus API Vertrag

```
publish(pattern: Pattern) -> bool, validiert gegen Schema Registry, ACL  
Check  
subscribe(filter: domain|pattern_type) -> stream<Pattern>  
ack(pattern_id, module_id) -> bool  
get_patterns(dag_id) -> list<Pattern>  
get_audit_log(dag_id) -> list<Pattern>
```

Bus selbst redundant: zwei Leitungen, wie CAN-Bus, mit eigenem Health Check. Rate Limit gegen Poisoning. Schema Validation.

Regeln:

- Schreiben nur im eigenen Namespace
- Lesen global aber nur getypte Felder
- Kein direkter Modul zu Modul Call
- TTL und Versionierung, automatische GC nach TTL
- Jeder Publish geloggt mit dag_id

7. Modul 4: Domänen Experten

7.1 Isolation

Jedes Experten-Modell: eigene Gewichte, eigene Loss Funktion, eigene Evaluierung, eigene Hardware. Kein Shared Encoder.

- Physik/Mathe Experte: Loss = numerische Stabilität + Einheitenkorrektheit + Formel Konsistenz. Trainiert auf Lehrbüchern, Papers, Simulationen.
- Code/Logik Experte: Loss = Syntax Validität + Unit Test Pass Rate + Typ Korrektheit. Trainiert auf Code + Tests.
- Sprache/Struktur Experte: Loss = semantische Kohärenz + Format Treue Markdown/JSON.
- Ethik/Logik Experte: Loss = Regelkonformität + Konsistenz.

7.2 Experten Output Format

$$E_i(T_i) = (y_i, c_i, t_i, model_{hash}, provenance)$$

```
{
  "node_id": "T_physik_01",
  "module_id": "physik_experte_v2.1",
  "model_hash": "sha256:physik_v2.1",
  "y_i": {
    "Q_dot": 15000,
    "delta_T": 25,
    "h_coefficient": 1200,
    "unit": "W/m2K",
    "formula": "Q_dot = h * A * delta_T"
  },
  "c_i": 0.94,
  "c_i_calibrated": 0.91,
  "ece_score": 0.03,
  "t_i_ms": 620,
  "provenance": {
    "used_patterns": ["CLOSED_LOOP_01"],
    "input_refs": ["user_request"],
    "blackboard_reads": 1
  }
}
```

Confidence Kalibrierung: Platt Scaling auf Validierungsset, Ziel ECE < 0.05. Ohne Kalibrierung ist Delta Prüfung wertlos.

7.3 Determinismus pro Experte

- Temperatur 0, top_p 1.0, top_k 1, fester Seed
- Reproduzierbare CUDA Kernel, deterministische Algorithmen
- Output validiert gegen `output_spec.schema_ref` vor Weitergabe. Bei Schema Verletzung: Retry mit Constraint.

8. Modul 5: Deterministische Fusions-Schicht

8.1 Aufgabe

Fusion darf nicht konkatinieren. Sie muss Widersprüche auflösen wie Endmontage in Automobil Fabrik. Physikalische und logische Kompatibilität erzwingen.

8.2 Konflikt Taxonomie

- TYPE_MISMATCH: int vs float64
- UNIT_MISMATCH: W vs kW, Celsius vs Kelvin
- CONFIDENCE_GAP: $|c_{\text{physik}} - c_{\text{code}}| > 0.25$
- LOGIC_CONTRADICTION: Formel sagt A, Code implementiert B
- FORMAT_VIOLATION: Output nicht gegen Schema valide

8.3 Fusions Algorithmus vollständig

```
function fuse(expert_outputs: list<E_i>, dag_id):
    delta_conf = max(c_i) - min(c_i)
    type_ok = validate_all_against_schema(expert_outputs)
    unit_ok = validate_units(expert_outputs)
    logic_ok = check_logic_consistency(expert_outputs)

    if delta_conf > 0.25 or !type_ok or !unit_ok or !logic_ok:
        constraint = {
            type: detect_conflict_type(),
            detail: "Variablen-Typisierung im Code verletzt physikalische
Fließkomma-Präzision: float64 erforderlich, int gefunden. Pattern
CLOSED_LOOP_01 erfordert while mit Konvergenz.",
            expected: load_schema("physik_output_v1.json"),
            actual: expert_outputs.code.y_i,
            delta: delta_conf,
            dag_id: dag_id,
            failing_module: "code_experte"
        }
        log_audit(event="fusion_constraint", constraint=constraint,
dag_id=dag_id)

        if iteration_count < MAX_ITER=3:
            // nur fehlerhaftes Modul neu triggern mit Constraint als
zusätzlichem Input
            trigger_rerun(module=constraint.failing_module,
extra_input=constraint, budget_increase=20%)
            iteration_count++
            return fuse(new_outputs) // rekursiv
        else:
            // Abbruch nach 3 Iterationen
            return {
                answer: best_effort_merge(expert_outputs),
                flag: "NON_CONVERGENT",
                provenance: collect_provenance(expert_outputs),
                audit: get_audit_log(dag_id),
                constraints_tried: all_constraints
            }
    else:
        return {
            answer: merge_with_template(expert_outputs),
            flag: "CONVERGENT",
            provenance: collect_provenance(expert_outputs),
            audit: get_audit_log(dag_id),
```

```
confidence_final: mean(c_i_calibrated)
}
```

Merger: Template basiert, deterministisch, kein freies Generieren. Temperatur 0, Seed fix.

8.4 Finale Antwort Format

```
{
  "dag_id": "dag_20261006_123456",
  "final_answer": "... synthetisierte Antwort mit Formeln und Code ...",
  "flag": "CONVERGENT",
  "confidence_final": 0.92,
  "provenance_graph": {
    "router_votes": ["router_a", "router_b"],
    "dag_hash": "sha256:d4e5",
    "experts_used": [
      {"module": "physik_experte_v2.1", "hash": "sha256:...", "c": 0.91},
      {"module": "code_experte_v3.0", "hash": "sha256:...", "c": 0.93}
    ],
    "blackboard_patterns": ["CLOSED_LOOP_01"],
    "fusion_iterations": 1,
    "model_hashes": ["sha256:..."]
  },
  "audit_log_ref": "s3://audit/dag_20261006_123456.jsonl",
  "generated_at": "2026-10-06T12:01:00Z"
}
```

9. Vollständige Datenmodelle und JSON Schemas

Verzeichnis /schemas/ :

- router_output_v1.json - Router Output
- dag_node_v1.json - DAG Node Spec
- blackboard_pattern_v1.json - Blackboard Pattern
- expert_output_v1.json - Experten Output E_i
- physik_output_v1.json - Domänen spezifisch
- code_output_v1.json
- fusion_constraint_v1.json
- final_answer_v1.json
- audit_event_v1.json

Alle Schemas versioniert, in Schema Registry, validiert vor jedem Publish.

Beispiel `expert_output_v1.json` :

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required":
  ["node_id", "module_id", "model_hash", "y_i", "c_i", "c_i_calibrated", "t_i_ms", "provenance"],
  "properties": {
    "node_id": {"type": "string"},
    "module_id": {"type": "string"},
    "model_hash": {"type": "string", "pattern": "^sha256:"},
    "y_i": {"type": "object"},
    "c_i": {"type": "number", "minimum": 0, "maximum": 1},
    "c_i_calibrated": {"type": "number"},
    "t_i_ms": {"type": "integer"},
    "provenance": {"type": "object"}
  }
}
```

10. Algorithmen und Pseudocode

Siehe Module 4,5,8. Zusätzlich:

- **DAG Builder Algorithmus:** Topologische Sortierung, Zyklus Erkennung, Budget Verteilung nach Priorität
- **ECE Kalibrierung:** Binning Methode, Platt Scaling Training monatlich
- **LoRA Training:** Rank 8-16, nur auf Failure Dataset, Early Stopping bei ECE Anstieg

11. Hardware Mapping und Kostenmodell

Modul	Hardware	vCPU	RAM	GPU	Kosten pro 1k Requests	Skalierung
TMR Router	Edge TPU / NPU	2	4GB	NPU	0.02€	Horizontal per Request
Physik Experte	H100 80GB	8	32GB	1x H100	0.45€	Queue + Autoscale 0-10
Code Experte	A6000 48GB	4	16GB	1x A6000	0.12€	Parallel
Sprache Experte	CPU / T4	4	8GB	0-1x T4	0.03€	Edge
Fusion Layer	A10 24GB	4	16GB	1x A10	0.05€	Single
Blackboard / Bus	CPU	2	8GB	0	0.01€	Redundant 2x

Vorteil: Einfache Textanfrage kostet nur Router + Sprache = 0.05€ statt 0.62€ für volle Pipeline. Strom und Compute fließen nur dorthin wo Router freigibt.

12. Selbstoptimierung - RL Loop und Continual Learning

12.1 Logging

Jeder DAG Pfad geloggt: dag_id, Router Votes, Decomposer Output, Blackboard Patterns, E_i mit c_i, t_i, Fusion Constraints, Iterationen.

Speicherung: JSONL in S3, OpenTelemetry Trace.

12.2 Trigger für Training

- Wenn c_i sinkt bei gleicher t_i über 100 Requests -> Drift
- Wenn Mikro-Iterationen für Modul C > 2 im Durchschnitt -> Halluzination
- Wenn TMR disagreement > 10% -> Router veraltet
- Wenn type_check Fail Rate > 2% -> Schema oder Modell veraltet

12.3 Training Ablauf

1. Failure Log -> dataset_modul_C_failures_v{n}.jsonl mit Constraint als Label und korrektem Output als Target
2. Shadow Training isoliert: LoRA Rank 8-16, nur Modul C, kein globales Training
3. Eval gegen Modul-eigenes Golden Benchmark + ECE Check
4. Canary Deployment: 5% Traffic auf neue LoRA Version, Monitoring 1h
5. Bei Erfolg: Model Registry Update, alter Hash archiviert, neuer Hash aktiv, Audit Eintrag
6. Rest des Clusters bleibt voll zertifiziert, keine globale Re-Zertifizierung
7. Bei Failure: Auto Rollback auf letzten guten Hash, Alert

Damit endet Re-Zertifizierungs Hölle. Update ist lokal, nicht global. LRU Prinzip.

13. Observability, Logging und Metriken

Metriken als Prometheus:

- dca_router_disagreement_rate
- dca_router_latency_p95_ms
- dca_dag_decomposition_latency_ms
- dca_blackboard_publish_rate_per_dag
- dca_blackboard_ttl_expired_total
- dca_fusion_convergence_rate
- dca_fusion_iterations_histogram
- dca_expert_ece_score{module="physik"}
- dca_expert_latency_p95_ms{module="code"}
- dca_expert_failure_rate{module="code", type="type_mismatch"}

Logging: JSONL mit Feldern timestamp, dag_id, event_type, module_id, model_hash, latency, confidence, constraint.

Dashboard: Grafana mit DAG Trace View.

14. Sicherheit, Isolation und Signierung

- Jede Experten Einheit in eigener Sandbox, Docker + gVisor, keine Netzwerkfreigabe außer Bus Port
- Modelle signiert mit Cosign, Hash verifiziert vor Inferenz. Kein unsigned Modell darf starten.
- Input Validation vor Router: Prompt Injection Filter, PII Filter, Max Token Check
- Blackboard: Schema Validation, ACL, Rate Limit 100 publishes per dag_id gegen Poisoning
- Fusion kann Experten hart abschalten bei $c_i < 0.2$ oder wiederholtem Fail
- Audit Log unveränderbar, WORM Storage S3 Object Lock
- Secrets: Model Registry Credentials via Vault, kein Hardcoding

15. Compliance Mapping EU AI Act

- Art. 9 Risikomanagement: Modulare Risikoanalyse pro Knoten, Failure Modes dokumentiert
- Art. 10 Daten Governance: Training Data Hashes geloggt, Failure Datasets versioniert
- Art. 11 Technische Dokumentation: Dieses Dossier + Provenance Graph + Model Hashes + Schemas
- Art. 12 Logging: Vollständiger Audit Log pro DAG, unveränderbar
- Art. 13 Transparenz: Finale Antwort enthält Flag CONVERGENT/NON_CONVERGENT und Confidence
- Art. 14 Menschliche Aufsicht: REVIEW_REQUIRED und NON_CONVERGENT gehen an Human-in-the-Loop Queue
- Art. 15 Genauigkeit und Robustheit: ECE Monitoring, Canary, Rollback

16. Deployment, Lifecycle und Model Registry

- **Model Registry:** MLflow oder HF Hub Enterprise. Speichert Model Hash, Training Data Hash, Eval Report, Signatur, LoRA Base Model.
- **CI/CD:** GitHub Actions: Lint Schemas, Unit Tests pro Modul, Golden Benchmark Eval, Sign, Push Registry.
- **Deployment:** Kubernetes. Jede Modul Art eigenes Deployment mit HPA. Blue/Green pro Modul. Rollback < 30s.
- **Versionierung:** SemVer pro Modul, z.B. `physik_experte v2.1.3`. DAG Schema Versionierung separat `dag_schema v1.0`. Kompatibilität Matrix.
- **Edge:** Router + Sprache lokal auf Gerät, HPC Module on-demand in Cloud via mTLS.

17. Failure Modes und Recovery - Vollständig

Failure	Erkennung	Recovery	Audit
TMR total disagreement	Kein Majority Hash	Flag REVIEW_REQUIRED, Human Review, Router Retraining Dataset	tmr_disagreement
Blackboard TTL expired	Pattern nicht acked in TTL	Re-publish Trigger, Alert wenn > 3x	ttl_expired
Fusion Deadlock > 3 Iter	MAX_ITER erreicht	NON_CONVERGENT + best effort + alle Constraints	fusion_deadlock
Experten Timeout	$t_i > \text{timeout_ms}$	Retry mit +20% Budget, dann Fallback Modul oder degraded mode	expert_timeout
LoRA Drift ECE steigt	$ECE > 0.08$ nach Training	Auto-Rollback auf letzten guten Hash, Alert, Dataset Review	lora_drift
Schema Verletzung	JSON Schema Validierung fail	Retry mit Constraint "Schema verletzte Feld X", dann Flag	schema_violation
Model Hash mismatch	Signatur ungültig	Pod startet nicht, Alert, Registry Check	security_violation

18. Implementierungs-Roadmap MVP bis Production

Phase 0 - MVP 2 Wochen:

- 1 Router statt TMR, Decomposer hart kodiert für 3 Tasks
- 2 Experten: Code + Sprache, Blackboard als In-Memory Dict
- Fusion simpel: Merge ohne Delta Check
- Ziel: End-to-End DAG läuft

Phase 1 - Determinismus 4 Wochen:

- JSON Schemas + Validierung
- Confidence Kalibrierung + ECE Messung
- Fusion mit Delta Prüfung und 1 Retry
- Audit Log JSONL + dag_id

Phase 2 - TMR + Bus 6 Wochen:

- 3 Router mit Voting, architektonische Diversität
- Blackboard mit Redis + Schema Registry + ACL
- Hardware Mapping + Budgeting
- Prometheus Metriken + Grafana

Phase 3 - Self-Optimization 8 Wochen:

- Model Registry + Signierung
- LoRA Training Pipeline + Canary + Rollback
- Failure Modes implementiert
- EU AI Act Doku und Human-in-the-Loop Queue

Phase 4 - Production:

- Autoscaling HPA, WORM Audit Storage, Edge Deployment, Kosten Dashboard

19. Glossar

- **DCA-01:** Deterministic Cluster Architecture Version 01
- **TMR:** Triple Modular Redundancy
- **DAG:** Directed Acyclic Graph
- **LRU:** Line Replaceable Unit
- **ECE:** Expected Calibration Error
- **LoRA:** Low-Rank Adaptation
- **Provenance:** Herkunftsnachweis welche Modelle und Patterns genutzt wurden
- **Blackboard:** Zentrales Shared Context Board
- **Deterministisch:** Gleicher Input + gleicher Model Hash + gleicher Seed = gleicher Output

20. Anhänge

Anhang A: Verzeichnisstruktur Repo

```
/dca-01/  
  /schemas/  
    router_output_v1.json  
    dag_node_v1.json  
    blackboard_pattern_v1.json  
    expert_output_v1.json  
    physik_output_v1.json  
    code_output_v1.json  
    fusion_constraint_v1.json  
    final_answer_v1.json  
    audit_event_v1.json  
  /src/  
    /router_tmr/  
    /decomposer/  
    /blackboard/  
    /experts/  
    /fusion/  
  /registry/  
  /eval/  
  DCA-01-COMPLETE.md  
  README.md  
  docker-compose.yml
```

Anhang B: Mermaid Diagramme

```
graph TD
  A[User Request] --> B[TMR Router]
  B -->|2-aus-3 Voting| C[Decomposer DAG]
  C --> D[Blackboard Bus]
  D --> E[Physik Experte]
  D --> F[Code Experte]
  D --> G[Sprache Experte]
  E --> H[Fusion Layer]
  F --> H
  G --> H
  H -->|Delta Check| I{Convergent?}
  I -->|Ja| J[Finale Antwort CONVERGENT]
  I -->|Nein <3 Iter| K[Constraint + Re-Run]
  K --> D
  I -->|Nein >=3| L[NON_CONVERGENT + Audit]
```

Anhang C: Eval Benchmarks pro Experte

- Physik: 500 Fragen Thermodynamik mit Einheiten Check, Ziel Genauigkeit > 92%, ECE < 0.05
- Code: HumanEval + eigene Unit Tests, Ziel Pass Rate > 95%
- Router: Golden Set 1000 Intents, Ziel Precision > 95%, Disagreement < 5%

Ende Dossier v1.2 COMPLETE

Diese Spezifikation ist vollständig. Sie enthält alle Verträge, Schemata, Algorithmen, Failure Handling, Security, Compliance und Lifecycle, um DCA-01 ohne weitere Rückfragen zu implementieren. Von Idee zu baufertiger Infrastruktur.

Autor: Emanuel Schaaf

Lizenz: Open Spec für industrielle Umsetzung